StockPilot

Gestion de stock pour une petite boutique
Soumission 1 : proposition et premier prototype

Langage de script - 420-LS3-AS
Automne 2026
Groupe : à compléter

Membres
Ilyas Safour
Zakaria Aouinat
Hajar Sellab

1. Problème et utilisateurs

Une petite boutique qui suit ses quantités sur papier risque de découvrir une rupture trop tard ou de déduire une quantité supérieure au stock disponible. Les employés ont besoin de connaître le stock courant et les produits à réapprovisionner. Le responsable pourra utiliser ces informations pour préparer ses commandes.

Nous choisissons ce sujet parce qu'il répond à un besoin concret, permet des calculs et validations simples, et peut évoluer progressivement pendant la session. La démonstration utilisera uniquement des produits et des quantités fictifs.

2. Portée du projet

Le projet concerne une seule boutique et des quantités entières en unités. Chaque produit possède un identifiant unique, un nom, une quantité et un seuil d'alerte. Une alerte apparaît lorsque la quantité est inférieure ou égale au seuil. Le stock ne peut jamais devenir négatif.

Les ventes, paiements, comptes utilisateurs, accès simultanés et intégrations commerciales sont hors de la portée prévue. Le premier prototype fonctionne en console et conserve les données uniquement en mémoire.

3. Fonctionnalités par priorité

Priorité	Fonctionnalité	État / étape
1	Consulter les identifiants, noms, quantités et seuils.	Réalisée en soumission 1
2	Enregistrer un mouvement : sortie ou entrée de stock.	Sortie réalisée ; entrée prévue en soumission 2
3	Afficher les produits dont quantité <= seuil.	Réalisée en soumission 1
4	Ajouter un produit avec un identifiant unique.	Prévue en soumission 2
5	Consulter l'historique des mouvements acceptés.	Prévue en soumission 2

4. Entrées, traitements et sorties

Entrées	Traitements	Sorties
Choix du menu : 0, 1, 2 ou 3.	Comparer le choix aux options proposées.	Écran demandé ou message de choix invalide.
Produits fictifs initialisés en mémoire.	Parcourir les produits ; comparer quantité et seuil.	Tableau du stock et état de chaque produit.
Identifiant et quantité de sortie.	Nettoyer l'identifiant ; chercher le produit ; convertir la quantité en entier ; vérifier quantité > 0 et quantité <= stock.	Message de refus ou confirmation avec stock restant.
Quantité disponible et seuil.	Filtrer les produits avec quantité <= seuil.	Liste des produits à commander.

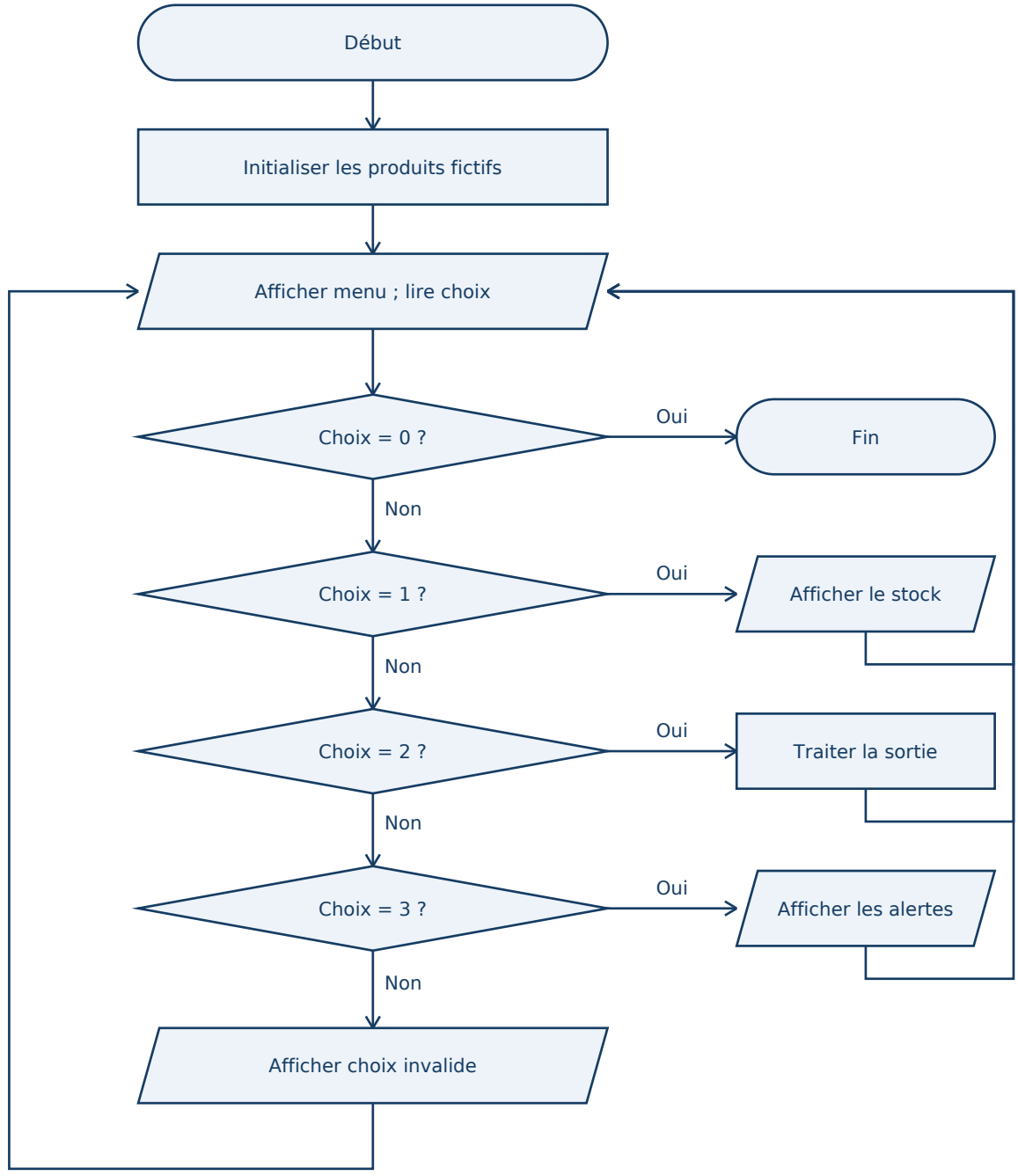
5. Organisation du prototype

main.py contient les fonctions creer_stock, afficher_stock, sortir_stock et main. Les produits sont stockés dans un dictionnaire indexé par identifiant. Les validations précèdent toute modification. test_stock.py exécute les scénarios automatiquement avec un stock neuf pour chaque cas.

Exemple initial : P001 / Cahier / 20 unités / seuil 5. La sortie de 15 unités laisse 5 unités et déclenche une alerte. Une sortie de 21 unités est refusée sans modifier le stock.

6. Organigramme général

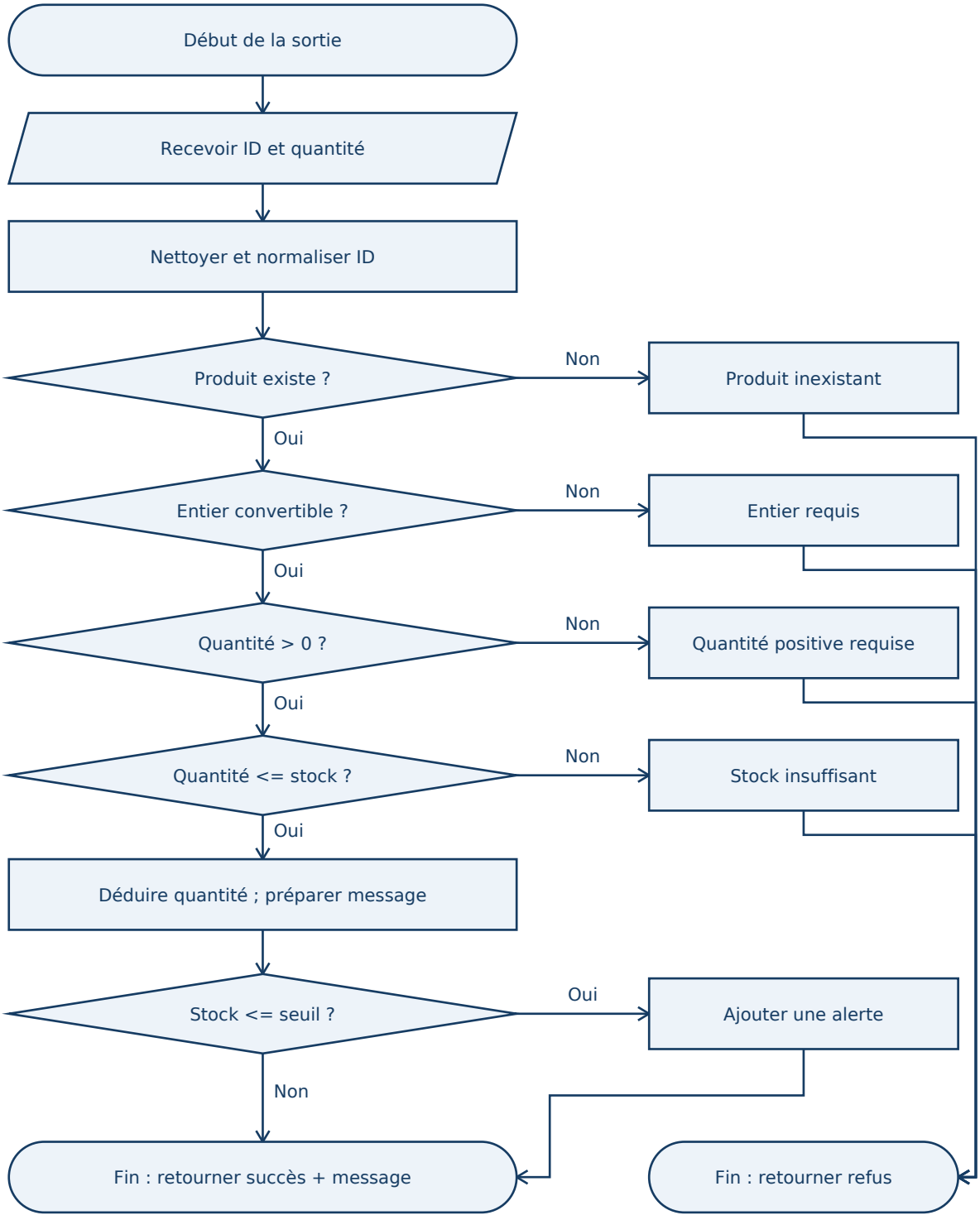
Le menu se répète après chaque consultation, traitement ou erreur. Le choix 0 termine le programme. Le traitement d’une sortie est détaillé à la page suivante.



Ovale : début / fin | Parallélogramme : entrée / sortie | Rectangle : traitement | Losange : décision

7. Organigramme de la sortie de stock

Les branches de refus quittent la fonction sans modification du stock. Le programme affiche ensuite le message retourné et revient au menu.



Ovale : début / fin | Parallélogramme : entrée / sortie | Rectangle : traitement | Losange : décision

8. Algorithme de la fonctionnalité centrale

1. Recevoir le stock, l’identifiant saisi et la quantité saisie.

2. Supprimer les espaces autour de l’identifiant et le convertir en majuscules.

3. Si le produit n’existe pas, retourner un refus : « Produit inexistant ».

4. Essayer de convertir la quantité en entier. Si la conversion échoue, retourner un refus : « La quantité doit être un entier ».

5. Si la quantité est inférieure ou égale à zéro, retourner un refus.

6. Si la quantité dépasse le stock du produit, retourner un refus : « Stock insuffisant ».

7. Déduire la quantité demandée du stock disponible.

8. Préparer la confirmation avec le stock restant.

9. Si le stock restant est inférieur ou égal au seuil, ajouter une alerte à la confirmation.

10. Retourner le succès et le message.

9. Stratégie de tests

Chaque cas démarre avec les mêmes données fictives : P001 contient 20 unités, seuil 5. Les tests comparent le succès ou le refus, le message exact et la quantité restante. Pour tous les refus, le dictionnaire complet doit rester identique à son état initial.

Commande : python test_stock.py. Les 11 cas ont été exécutés dans l’environnement de préparation et sont réussis. Le fichier docs/resultats_tests.txt conserve les résultats observés. Ils devront aussi être reproduits sur les ordinateurs de l’équipe avant la remise.

Une démonstration du menu a également été réalisée : consultation, sortie jusqu’au seuil, refus d’une quantité excessive, affichage des alertes et fermeture.

10. Cas de test et résultats

Cas / type	Entrées	Résultat attendu	Résultat observé / verdict
T01 / Normal	P001, '4'	Acceptée ; P001 = 16. Message : Sortie enregistrée. Stock restant : 16.	P001 = 16. Sortie enregistrée. Stock restant : 16. Verdict : RÉUSSI.
T02 / Limite : stock épuisé	P001, '20'	Acceptée ; P001 = 0. Alerte. Message : Sortie enregistrée. Stock restant : 0. Alerte : produit à commander.	P001 = 0. Sortie enregistrée. Stock restant : 0. Alerte : produit à commander. Verdict : RÉUSSI.
T03 / Limite : seuil atteint	P001, '15'	Acceptée ; P001 = 5. Alerte. Message : Sortie enregistrée. Stock restant : 5. Alerte : produit à commander.	P001 = 5. Sortie enregistrée. Stock restant : 5. Alerte : produit à commander. Verdict : RÉUSSI.
T04 / Invalide : stock insuffisant	P001, '21'	Refusée ; P001 = 20. Stock inchangé. Message : Stock insuffisant : sortie refusée.	P001 = 20. Stock insuffisant : sortie refusée. Verdict : RÉUSSI.
T05 / Invalide : zéro	P001, '0'	Refusée ; P001 = 20. Stock inchangé. Message : La quantité doit être supérieure à zéro.	P001 = 20. La quantité doit être supérieure à zéro. Verdict : RÉUSSI.
T06 / Invalide : négatif	P001, '-2'	Refusée ; P001 = 20. Stock inchangé. Message : La quantité doit être supérieure à zéro.	P001 = 20. La quantité doit être supérieure à zéro. Verdict : RÉUSSI.
T07 / Invalide : texte	P001, 'abc'	Refusée ; P001 = 20. Stock inchangé. Message : La quantité doit être un entier.	P001 = 20. La quantité doit être un entier. Verdict : RÉUSSI.
T08 / Invalide : décimal	P001, '2.5'	Refusée ; P001 = 20. Stock inchangé. Message : La quantité doit être un entier.	P001 = 20. La quantité doit être un entier. Verdict : RÉUSSI.
T09 / Invalide : produit absent	P999, '2'	Refusée ; P001 = 20. Stock inchangé. Message : Produit inexistant.	P001 = 20. Produit inexistant. Verdict : RÉUSSI.
T10 / Normal : espaces et minuscules	p001 , ' 4 '	Acceptée ; P001 = 16. Message : Sortie enregistrée. Stock restant : 16.	P001 = 16. Sortie enregistrée. Stock restant : 16. Verdict : RÉUSSI.
T11 / Invalide : saisie vide	P001, ''	Refusée ; P001 = 20. Stock inchangé. Message : La quantité doit être un entier.	P001 = 20. La quantité doit être un entier. Verdict : RÉUSSI.

11. Prochaines étapes : soumission 2

Ajouter les entrées de stock, l’ajout de produits et l’historique. Sauvegarder les produits et mouvements au format JSON, puis les recharger au lancement. Gérer les fichiers absents ou invalides avec des messages explicites. Étendre les tests et rédiger le rapport de progression de 2 à 4 pages demandé. Adapter le projet à la rétroaction du professeur.

Pour la finale, l’équipe prévoit une interface Tkinter et une classe Produit, si ces choix restent adaptés aux notions vues en classe.

12. Collaboration et aide reçue

Chaque membre relira le dossier, exécutera les tests et pourra expliquer la sortie de stock. Les contributions seront consignées dans Git au fur et à mesure. La répartition proposée est : Ilyas, prototype et intégration ; Zakaria, cas de test et vérification ; Hajar, besoin et organigrammes. Cette répartition reste à confirmer par l’équipe.

Une IA générative a aidé à préparer ce dossier, le prototype et les tests. L’équipe doit vérifier l’autorisation de cette aide, relire le travail, l’adapter et être capable d’expliquer et de modifier chaque partie du code.

13. Conditions de validation de la première étape

Le prototype doit démarrer sans dépendance externe, permettre une sortie valide et refuser toute sortie invalide sans modifier les données. L’algorithme et les deux organigrammes doivent correspondre au fonctionnement démontré. Les cinq fonctionnalités proposées constituent une portée réaliste pour la session.

Limites actuelles : trois produits fictifs prédéfinis, aucune sauvegarde après fermeture et aucune utilisation simultanée. Ces limites sont assumées pour le premier prototype. La persistance sera développée à la deuxième étape.